06-03-2026: Make this code the basis for julia port

Copy to fvh1_7. Flatten to a single directory (eliminate
/ppmlr directory), and adjust make file.

Copy to julia_fvh1_7 and start port. Debugging in
julia_fvh1_8

06-03-2026: Port to julia

Ported using copilot in vscode. Port in the (inverse) order
that functions are called.

0) modules: vh1mods, zone
1) basic functions: evolve, mc3d, init, stress, parabola,
   boundary, states, dtcon
2) next level: remap
3) next level: ppmlr
4) next level: sweepx, sweepy, sweepz
5) finaL; vhone, myprin

Debugging initial julia port using Cursor (06-05-2026)
---

1. Working directory (vhone.jl)
Replaced the stale cd("my_vh1_2026/julia_fvh1_7/julia_port")
with cd(@__DIR__) so the script works from any launch directory.

2. Output file types (vhone.jl, prin.jl)
open() returns IOStream, but myprin! expected Dict{Int, IO}. Fixed
by typing open_output_files() as Dict{Int,IO}(...) and using
Dict{Int,<:IO} in function signatures.

3. Integer conversion (vhone.jl)
Changed Int(...) to floor(Int, ...) for nprin, matching Fortran’s
truncating int() behavior.

4. Stale sweep indices (boundary.jl, ppmlr.jl, evolve.jl, remap.jl,
states.jl). Imported nmin/nmax/nleft/nright were frozen at 0 in Main
while Sweeps.nmin was correctly set to 7. Each affected function now
reads fresh values from Sweeps (and Global.dt in states.jl).

5. Get many warnings (etc)
WARNING: ignoring conflicting import of Zone.zro into Main
ChatGPT recomments:

```
using .Sweeps
using .Zone
Sweeps.nmin = 7
Zone.zro = 0
```

rather than

```
using .Sweeps: nmin
using .Zone: zro
```

Note: For debugging

```
   using Pkg; Pkg.add("Infiltrator")
and then
   using Infiltrator
   @infiltrate  # Execution will pause here
-------------------------------------------------------------------------

Make explicit reference to vars from modules
--------------------------------------------
Change

using .Zone : zro
zro[i,j,k] =

--> Using .Zone
    Zone.zro[i,j,k] =

and remove export in modules. Have changed
vhone.jl, init.jl, prin.jl, sweepx,jl, sweepy.jl, sweepz.jl
evolve.jl, stress.jl, boundary.jl, mc3d.jl, parabola.jl,
states.jl, dtcon.jl

Note: remove para from sweeps.jl. Explicitely define in
ppmlr.jl and remap.jl (because para is an argument in call
to parabola() and paraset()).

Minor errors
------------
Some references still incorrect (e.g. radius --> Sweeps.radius
in volume()). In vhone.jl calculation of cor[j,k] is missing
normalization by isample.

06-11-2026: Results agree with fvh1_7/vhone.f90
-----------------------------------------------
But, julia code has sizeable numerical errors. Consider
40^3 lattice, dt=0.05, tmax=5.

             fortran              julia
mass         6*10^(-6)%           1%
momentum     1.8*10^(-2)          312.1
energy       1.4*1-^(-4)%         1.6%


Consider pure ideal evolution

             fortran              julia
mass         2*10^(-4)            4*10^(-5)
momentum     0.1                  3.6
energy       2*10^(-4)            7*10^(-5)

Stochastic evolution only

             fortran              julia
mass         0%                   0%
momentum     3*10^(-4)            2*10^(-12)
energy       2*10^(-4)%           0%

Problem is is in parabola.jl. Bounds in loops that
compute al, ar, a6 not correctly copied from parabola.f90.

Fixes the issue. Now agrees with fvh1_7/vhone.f90
```

06-15-2026: introduce structs
---
copy to julia_fvh1_9. run the following prompt in cursor:

I want to remove the modules Global, Sweepsize, Sweeps,
and Zone from vhone.jl and the subroutines that are
called from vhone.jl.

1) Make the following variables constants with global scope:
gam, pi, gamm, alpha, alphat, nav, tav, etaav,
A0, V0, smallp, smallr, small,
uinflo,dinflo,vinflo,winflo,pinflo,einflo,
uotflo,dotflo,votflo,wotflo,potflo,eotflo.

2) Make a struct called globalvars that contains
ncycle, ncycp, ntrial, nacc
time, dt, timep, dtfix, endtime, tprin

3) Make maxsweep a global constant variable.

4) Make a struct called Sweeps that contains
nmin, nmax, nleft, nright,
r, p, e, q, u, v, w,
xa, xa0, dx, dx0, dvol, dvol0,
flat, radius, area
Use a struct of this type to pass these variables
to ppmlr(), boundary(), volume(), evolve(), remap(),
etc.

5) make imax,jmax,kmax, and ndim constant variables with global
scope.

6) make a struct called zone that contains
zro, zpr, zux, zuy, zuz
use this struct to call sweepx(),sweepy(),sweepz(),mc3d(),
prin(), etc.

7) make a struct called zonegrid that contains
zxa, zdx, zxc,
zya, zdy, zyc,
zza, zdz, zzc,
nleftx, nlefty, nleftz,
nrightx, nrighty, nrightz
use this struct to call sweepx(),sweepy(),sweepz(),mc3d(),
prin(), etc.

8) make a strcut called zoneobs that contains
repixky, impixky,
spec, spec2,
rhokx.

-> vh1mods.jl, zonemod.jl removed. Works fine

In cursor: How would you optimize this code?

2. Stop allocating in hot loops (biggest win)
The code allocates heavily inside the timestep loop.

mc3d! — worst offender. For each of 64,000 cells it allocates
iidx, lambda, cube_*, sign_*, pcube*, etc. That is millions of
small allocations per call, and you call it twice per cycle.

Fix:

Preallocate p1, p2, p3, q once in a McWork struct (or in zone).
Hoist the 8-corner cube geometry out of the (i,j,k) loop; only
the indices change.
Replace lambda + transpose(lambda) with in-place updates.
Unroll the 8-cube loop with direct indexing instead of cube_i[cube_idx].
ppmlr! / parabola! / remap! / evolve! / states! — each call allocates ~15–30
vectors of length maxsweep.
sweepx! calls ppmlr! 1,600 times per sweep (40×40 lines), and you do
6 sweeps per cycle.

Fix:

Add a PpmWork struct holding all scratch arrays.
Pass it into ppmlr!, parabola!, remap!, etc., and reuse every call.
That alone can easily give 5–20× on the hydro sweeps by eliminating GC.

3. Parallelize independent work
Sweeps: In sweepx!, each (j, k) line is independent. Parallelize
over j or (j,k):

```
Threads.@threads for j in 1:jmax
    local_sweeps = ...  # one Sweeps + PpmWork per thread
    ...
end
```

Each thread needs its own Sweeps and scratch buffers.

mc3d!: Each cell update is independent (stochastic). Parallelize
over (i,j,k) with thread-local RNGs (Random.Xoshiro per thread).

Use Threads.nthreads() and check scaling with @time before/after.

4. Reduce per-cycle overhead outside the physics
Every cycle currently does:

Full-grid energy!, entropy!, scalesize!, momentum!, pixky!
Multiple file writes
println every step
For production runs:

Compute diagnostics every N cycles, not every cycle.
Buffer I/O or write less often.
Remove or gate the println in the main loop.
That won't change physics cost much, but it matters when diagnostics
are a large fraction of runtime.

5. Low-level Julia tweaks (after allocation fixes)
Once buffers are reused:

Mark inner loops @inbounds @simd where safe.
Use @views to avoid copies when passing sweep slices.
Keep loop order i innermost for 3D arrays (column-major layout)

— you mostly already do this. Consider @fastmath in ppmlr! /
evolve! if you're OK with slightly looser floating-point semantics.
These are usually 10–30%, not order-of-magnitude, once allocation
is fixed.

6. Structural / algorithmic options

Change	Tradeoff
Smaller grid during development (imax=20, etc.)	Faster iteration, not a production speedup
@static grid sizes (Val{40}())	Better unrolling/constant propagation
GPU kernels for mc3d! or line sweeps	Large effort; best after CPU code is clean
Fuse the six sweeps	Hard; operator splitting is intentional

Suggested order of work
Add reusable work buffers for ppmlr! and friends.
Rewrite mc3d! to allocate nothing inside the (i,j,k) loop.
Profile again.

First optimization:

Implement step 1-2, store in julia_vh1_10

Check: julia_vh1_8_test, julia_vh1_9, julia_vh1_10
agree. vh_10 about 7x faster.

06-15-2026: parallel sweeps
---
copy to julia_fvh1_11.

sweepx!, sweepy!, sweepz! use Threads.@threads over independent
1-D lines (1600 lines per sweep direction). Each thread gets its
own Sweeps + PpmWork from a SweepThreadPool (sized to
Threads.maxthreadid()).

Run with e.g. JULIA_NUM_THREADS=4 julia run_vhone.jl

Sweep-only scaling (40^3 grid, 6 sweeps/cycle):
  1 thread: 0.132 s/cycle
  4 threads: 0.059 s/cycle (~2.2x)
  8 threads: 0.040 s/cycle (~3.3x)

Serial vh1_11 matches vh1_10 acceptance rate exactly.
Full endtime=5 run: ~27 s (4 threads) vs ~30 s (1 thread).

06-15-2026: parallel pixky!
---
pixky! uses Threads.@threads over Fourier modes (in_=1:nk). Each mode
accumulates independently over the grid — no shared writes, no reduction.
Trig factors precomputed once in PIXKY_COS_J, PIXKY_SIN_J, PIXKY_SIN_I.

06-15-2026: parallelize mc3d
---
cursor prompt: parallelize m3cd. Note that every corner has to be
touched, but each corner update involves 8 adjacent cells, and
we have to prevent updates from interfering.

Created /home/tmschaef/my_vh1_2026/julia_fvh1_12 by copying julia_fvh1_11
and implemented parallel mc3d! with an 8-color checkerboard (8 sequential

phases, Threads.@threads within each). Updated vh1state.jl (McThreadPool/McScratch), mc3d.jl, and vhone.jl. Full test with 4 threads and endtime=5 completed in 7 s with acceptance rate 0.672

On a 40^3 grid (Julia 1.12.5, 14 cores), Benchmark mc3d speedup measured 1.9× at 4 threads and 3.4× at 8 threads per mc3d! call (sequential baseline ~0.023 s). Scaling is sub-linear because of the 8 phase barriers and memory bandwidth; 14 threads was slightly slower than 8 (3.0×). No further action needed.

All draws inside mc3d_cell! go through that local RNG (gaussian_random(rng), rand(rng) for Metropolis), not the global rand().

Three caveats worth knowing:

*Not the same sequence as the old serial code. The serial version used one global stream in (i,j,k) order. The parallel version uses separate streams with different seeds, consumed in color-phase / thread order. Statistically fine for MC; not bit-for-bit reproducible vs serial.

Random.seed!(123456) in vhone.jl does not affect mc3d!. That seeds the global RNG; mc3d ignores it.

Run-to-run reproducibility within the parallel code depends on @threads scheduling. Each thread's stream is deterministic given a fixed draw count, but if thread 4 processes cell A before cell B in one run and the reverse in another, the mapping from grid cell -> random numbers can change. The streams remain independent; only the assignment of draws to cells may vary.

So: independent across threads, yes. Same as serial, or perfectly deterministic cell-by-cell across parallel runs — no, unless you add explicit per-cell seeding or a deterministic work partition.